FORGE

Robust and Selective Knowledge Unlearning in Large Language Models

Project Blueprint • Research Methodology • Novelty • Two-Contributor Execution Plan

Working research framework: FORGE
Target: Conference-quality empirical research paper
Primary benchmark: TOFU
Initial development model: TinyLlama 1.1B
Team: 2 contributors

1. Executive Summary

FORGE is a research project investigating whether machine unlearning in large language models (LLMs) actually removes targeted knowledge or merely suppresses its expression. The project focuses on selective forgetting: a model should forget specified knowledge while preserving unrelated and useful knowledge.

The central evaluation idea is to measure the collateral effects of targeted forgetting using a knowledge 'blast radius'. Rather than evaluating only whether a model fails on the exact forget questions, FORGE evaluates multiple ways that the forgotten knowledge may reappear—direct recall, paraphrased prompts, indirect inference, and relearning—and measures how far the effects of unlearning spread into semantically related and unrelated knowledge.

A key research-design principle is that the final unlearning method will not be fixed in advance. The team will first reproduce strong baselines, diagnose their failure modes, and then design a targeted improvement supported by evidence. This avoids presenting an arbitrary algorithm as a novel contribution.

2. Core Research Idea

2.1 Problem

LLMs can memorize or encode sensitive, copyrighted, outdated, or otherwise unwanted information. When a particular subset of knowledge must be removed, retraining the entire model from scratch is expensive. Machine unlearning seeks to remove the influence of selected data or knowledge while retaining the model's broader capabilities.

The difficulty is that successful unlearning has two dimensions: the target knowledge should become unavailable, and knowledge that was not requested for deletion should remain available. A method can appear successful under a narrow forget-set metric while damaging neighboring knowledge or leaving the target recoverable through another prompt.

2.2 Central Research Question

Do current LLM unlearning methods genuinely remove targeted knowledge, or do they primarily suppress its expression, and how can forgetting be made more robust while limiting collateral damage to unrelated knowledge?

2.3 Desired behavior

Knowledge region	After unlearning
Target knowledge	Strongly forgotten
Closely related knowledge	Preserved as much as possible
Moderately related knowledge	Minimal degradation
Unrelated/general knowledge	Essentially preserved

3. Project Goals

- Build a reproducible LLM unlearning experimental pipeline using TOFU.
- Establish a trustworthy pre-unlearning baseline before changing model weights.
- Reproduce and compare multiple strong unlearning baselines.
- Measure forgetting beyond exact-match/direct prompts.

- Test robustness to paraphrasing, indirect inference, and relearning.
- Quantify collateral knowledge damage using a semantic blast-radius analysis.
- Characterize the forgetting–utility/collateral-damage trade-off.
- Use observed weaknesses to formulate and test an improved unlearning method.
- Run ablations, multiple forget-set sizes, and statistical analysis where compute permits.
- Produce reproducible artifacts and a conference-quality research paper.

4. Research Methodology

4.1 Overall experimental pipeline

The project follows an evidence-driven sequence. The proposed method is intentionally delayed until baseline behavior has been measured.

1. Dataset verification and controlled forget/retain splits

2. Base-model loading and pre-unlearning evaluation

3. Baseline unlearning reproduction

4. Multi-dimensional robustness evaluation

5. Collateral-damage / blast-radius evaluation

6. Failure-mode diagnosis

7. Proposed method design

8. Ablation studies

9. Final experiments across forget percentages/seeds/models

10. Statistical analysis, plots, paper writing, and reproducibility packaging

4.2 Dataset

TOFU (Task of Fictitious Unlearning) is the primary benchmark. The verified local copy contains 4,000 question–answer examples in a train split. FORGE currently uses deterministic seed 42 and creates controlled forget sets of 1%, 5%, and 10%.

Experiment	Forget examples	Retain examples
1% forget	40	3,960
5% forget	200	3,800
10% forget	400	3,600

4.3 Development model

TinyLlama/TinyLlama-1.1B-Chat-v1.0 is the initial development model because it is substantially cheaper and faster to iterate on than 7B–8B models. It should be treated as a development and debugging model rather than automatically as the sole final validation model.

For the final paper, the team should validate the final method on a stronger and/or architecturally different open model if compute permits. This reduces the risk that the findings are specific to a single small model.

4.4 Evaluation dimensions

Dimension	Research question
Direct forgetting	Can the model still answer the original forget questions?
Paraphrase robustness	Does the forgotten information reappear under semantically equivalent prompts?
Indirect inference	Can the model reconstruct the target knowledge through related questions or reasoning?
Relearning resistance	Can a small amount of additional exposure quickly restore the forgotten knowledge?
Retention	Does unrelated knowledge remain usable?
Semantic collateral damage	How much performance degrades around the target knowledge?
Forgetting–utility trade-off	How much retention is sacrificed to achieve a given level of forgetting?

5. Blast Radius: Core Evaluation Lens

FORGE uses 'blast radius' as an evaluation dimension rather than claiming that the concept itself is novel. The purpose is to quantify how far model damage spreads beyond the intended forget target.

A semantic neighborhood can be organized from the target entity/fact outward: target knowledge → closely related knowledge → moderately related knowledge → unrelated knowledge. Performance can then be plotted against semantic distance to estimate the scope of collateral degradation.

A strong unlearning result should show a sharp effect at the target while keeping the surrounding regions comparatively stable. If degradation spreads broadly, the method has a large collateral blast radius even if its direct forget score looks strong.

6. Research Gaps and How FORGE Addresses Them

Gap	Why it matters	FORGE response
Narrow evaluation of forgetting	Many evaluations can emphasize direct forget-set behavior.	FORGE tests direct recall, paraphrases, indirect inference, and relearning so that apparent forgetting is challenged from multiple retrieval paths.
Suppression vs. genuine removal	A model may fail to answer a particular prompt while retaining recoverable representations of the knowledge.	FORGE explicitly probes alternate prompts and recovery/relearning behavior instead of treating one failed response as proof of erasure.
Collateral damage is underemphasized in simple pipelines	High target forgetting can be achieved at the cost of unrelated knowledge.	FORGE measures retention and semantic neighborhoods and reports a forgetting–collateral-damage trade-off.
Inconsistent robustness characterization	Different studies use different tests, making failure modes difficult to compare.	FORGE establishes a repeatable evaluation protocol covering several attack/recovery routes.
Method-first rather than evidence-first design	A proposed technique can be selected before understanding which failure mode actually dominates.	FORGE delays the proposed method until baseline diagnosis identifies a stable, measurable weakness.
Limited generalization evidence	Results on one model/configuration may not transfer to other models.	FORGE plans validation across additional model scale/architecture where compute permits.

Important novelty caution: adaptive learning rates, parameter localization, semantic collateral damage, and relearning robustness have all appeared in recent unlearning research. FORGE therefore should not claim any of these components as novel in isolation. The novelty should emerge from the empirically supported

combination of diagnosis, evaluation, and the final method designed to address the specific failure mode discovered in the experiments.

7. Expected Novel Contribution

The final contribution is intentionally conditional on experimental findings. A plausible direction, if baseline experiments show that direct forgetting is achieved but alternate retrieval/relearning remains effective, is to introduce an explicit robustness-aware unlearning objective while constraining retention loss. Conceptually, this may take the form of a forgetting objective plus retention and robustness terms.

This equation is a design hypothesis, not the final claimed contribution. The exact formulation, training procedure, and novelty claim must be determined after baseline results and literature verification.

The strongest final paper story will be: (1) identify a reproducible failure in existing methods; (2) explain why it occurs; (3) propose a targeted intervention; (4) show improved forgetting robustness with a smaller blast radius; and (5) demonstrate that the improvement survives ablations and, ideally, a second model.

8. Baseline and Comparison Strategy

The team should reproduce approximately 3–4 strong and representative methods rather than implementing a large number superficially. Candidate families include gradient-ascent-based unlearning, preference/loss-based methods such as NPO, representation-based approaches such as RMU where compatible, and a strong recent selective/localization method. Exact baseline selection should be finalized after checking code availability, model compatibility, and compute requirements.

Stage	Purpose
Base model	Establish original knowledge/utility before unlearning.
Baseline A	Representative optimization-based method.
Baseline B	Alternative objective / preference-based method.
Baseline C	Representation/localization-based method where feasible.
FORGE method	Address the empirically diagnosed failure mode.

9. Experimental Design for Publication Quality

- Forget sizes: 1%, 5%, and 10% as the initial controlled settings.
- Seed control: seed 42 for the core pipeline; additional seeds for important final experiments where compute permits.
- Metrics: report both forgetting and retention rather than optimizing only one number.
- Robustness: evaluate direct, paraphrase, indirect, and relearning behavior.
- Collateral damage: report performance across semantic-distance bands.
- Ablation: remove one proposed component at a time and test sensitivity to key hyperparameters.
- Statistical analysis: report mean/standard deviation or confidence intervals across repeated runs when feasible.
- Generalization: use a second model or model scale for the strongest final claims if resources allow.
- Reproducibility: store configs, seeds, evaluation prompts, checkpoints/metadata, and result tables; exclude large generated artifacts from Git.

10. Two-Contributor Work Allocation

Contributor 1 — Mihika	Contributor 2 — Research/Engineering Partner
Experiment orchestration, dataset pipeline, evaluation design, result analysis, paper integration	Baseline reproduction, training/unlearning implementation, model/compute setup, code review
Maintain experiment configs and result tables	Maintain unlearning modules and training scripts
Lead blast-radius and robustness analysis	Lead baseline correctness and computational reproducibility
Lead figures, statistical summaries, and paper narrative	Lead ablations, implementation documentation, and experiment logs

Both contributors should review major methodological decisions together. Each branch should contain small, reviewable commits. Merge only tested changes into main.

11. Day-by-Day Timeline — 2 Contributors

Recommended duration: approximately 12 weeks / 84 days. The plan is intentionally research-oriented rather than simply software-oriented. Some days are lighter or reserved for debugging, reruns, and analysis. If compute is limited, final-model validation can be compressed while preserving the core diagnostic experiments.

Day	Owner	Task	End-of-day deliverable
Day 1	Mihika	Finalize research question, scope, hypotheses, and success criteria.	Shared research brief v1
Day 1	Contributor 2	Review recent unlearning literature and shortlist candidate baselines.	Baseline shortlist
Day 2	Mihika	Organize literature matrix: method, model, dataset, metric, weakness.	Literature matrix
Day 2	Contributor 2	Check baseline repositories/papers for model and dataset compatibility.	Compatibility notes
Day 3	Mihika	Finalize project structure, configs, seeds, and experiment naming conventions.	Experiment protocol
Day 3	Contributor 2	Verify compute options and estimate runtime/memory for candidate models.	Compute plan
Day 4	Mihika	Verify TOFU loader and dataset statistics.	Verified TOFU loader
Day 4	Contributor 2	Review dataset splitting methodology and reproducibility requirements.	Split review
Day 5	Mihika	Generate deterministic 1%, 5%, 10% forget/retain splits.	Processed split pipeline
Day 5	Contributor 2	Validate split sizes, overlap, and seed reproducibility.	Split validation report
Day 6	Mihika	Document dataset and experimental protocol.	Dataset protocol
Day 6	Contributor 2	Set up model loading and generation tests.	Model setup script
Day 7	Mihika	Run base-model sanity checks and inspect representative TOFU prompts.	Baseline sanity log
Day 7	Contributor 2	Confirm model/tokenizer versions and hardware behavior.	Environment lock
Day 8	Mihika	Design baseline evaluation metrics and result schema.	Evaluation specification
Day 8	Contributor 2	Implement common	Evaluation utilities

		inference/evaluation utilities.	
Day 9	Mihika	Implement pre-unlearning direct evaluation.	Pre-unlearning evaluator
Day 9	Contributor 2	Test evaluator on a small subset and verify outputs.	Evaluator validation
Day 10	Mihika	Run full pre-unlearning baseline on 1% split.	Baseline results v1
Day 10	Contributor 2	Audit baseline outputs and metric calculations.	Metric audit
Day 11	Mihika	Extend baseline to 5% and 10%.	Baseline results v2
Day 11	Contributor 2	Create standardized experiment runner/config loading.	Experiment runner
Day 12	Mihika	Build paraphrase evaluation protocol.	Paraphrase evaluator
Day 12	Contributor 2	Build indirect-inference prompt/evaluation utilities.	Indirect evaluator
Day 13	Mihika	Design relearning experiment protocol.	Relearning protocol
Day 13	Contributor 2	Implement controlled relearning procedure.	Relearning module
Day 14	Mihika	Run robustness sanity experiments on base model.	Robustness baseline
Day 14	Contributor 2	Debug and validate all robustness scripts.	Robustness QA
Day 15	Mihika	Select final 3–4 baselines with literature justification.	Baseline final list
Day 15	Contributor 2	Prepare/port baseline implementations.	Baseline code skeletons
Day 16	Mihika	Document baseline hyperparameters from source papers/repos.	Baseline config sheet
Day 16	Contributor 2	Run first baseline on small subset.	Baseline smoke test
Day 17	Mihika	Verify baseline outputs against expected qualitative behavior.	Baseline correctness report
Day 17	Contributor 2	Fix training/unlearning implementation issues.	Corrected baseline
Day 18	Mihika	Run Baseline A on 1% forget set.	A-1% results
Day 18	Contributor 2	Monitor runtime, memory, checkpointing.	Run log
Day 19	Mihika	Run Baseline A on 5% and 10%.	A results
Day 19	Contributor 2	Validate reproducibility and checkpoints.	A validation
Day 20	Mihika	Evaluate Baseline A for paraphrase/indirect/relearning.	A robustness results
Day 20	Contributor 2	Run retention evaluation for A.	A retention results
Day 21	Mihika	Analyze A forgetting–retention trade-off.	A analysis
Day 21	Contributor 2	Code review and experiment cleanup.	A clean pipeline
Day 22	Mihika	Run Baseline B smoke test.	B smoke results
Day 22	Contributor 2	Fix/validate Baseline B.	B validated
Day 23	Mihika	Run Baseline B across forget sizes.	B results
Day 23	Contributor 2	Run retention and robustness evaluation.	B evaluation
Day 24	Mihika	Analyze B results.	B analysis
Day 24	Contributor 2	Cross-check metrics and logs.	B audit
Day 25	Mihika	Run Baseline C smoke test.	C smoke results
Day 25	Contributor 2	Fix/validate Baseline C.	C validated
Day 26	Mihika	Run C across forget sizes.	C results
Day 26	Contributor 2	Run retention/robustness evaluation.	C evaluation

Day 27	Mihika	Analyze C results.	C analysis
Day 27	Contributor 2	Create unified baseline results table.	Baseline table v1
Day 28	Mihika	Compare baselines on direct forgetting and retention.	Comparison analysis
Day 28	Contributor 2	Compare computational cost and stability.	Efficiency analysis
Day 29	Mihika	Run full paraphrase comparisons.	Paraphrase comparison
Day 29	Contributor 2	Run indirect-inference comparisons.	Indirect comparison
Day 30	Mihika	Run relearning comparisons.	Relearning comparison
Day 30	Contributor 2	Verify all evaluation seeds/configs.	Evaluation audit
Day 31	Mihika	Create semantic neighborhood design for blast-radius evaluation.	Neighborhood protocol
Day 31	Contributor 2	Implement similarity/semantic-distance utilities.	Semantic utility
Day 32	Mihika	Construct target/related/moderate/unrelated evaluation groups.	Blast-radius dataset
Day 32	Contributor 2	Validate grouping methodology and leakage risks.	Grouping validation
Day 33	Mihika	Run blast-radius evaluation for strongest baseline.	Blast-radius results A
Day 33	Contributor 2	Run same for second baseline.	Blast-radius results B
Day 34	Mihika	Run blast-radius evaluation for remaining baseline.	Blast-radius results C
Day 34	Contributor 2	Audit semantic evaluation and metrics.	Blast-radius audit
Day 35	Mihika	Plot degradation vs semantic distance.	Blast-radius plots
Day 35	Contributor 2	Compute aggregate collateral-damage metrics.	Collateral metrics
Day 36	Mihika	Integrate forgetting, robustness, retention, collateral metrics.	Unified scorecard
Day 36	Contributor 2	Cross-check all result tables.	Results audit
Day 37	Mihika	Identify dominant baseline failure mode.	Failure hypothesis
Day 37	Contributor 2	Inspect training dynamics/checkpoints for mechanism clues.	Mechanism analysis
Day 38	Mihika	Test whether failure persists across forget sizes.	Failure replication
Day 38	Contributor 2	Test sensitivity to hyperparameters.	Sensitivity results
Day 39	Mihika	Test failure under alternate prompts/retrieval paths.	Robustness diagnosis
Day 39	Contributor 2	Test recovery/relearning behavior.	Recovery diagnosis
Day 40	Mihika	Write precise research gap statement from evidence.	Gap statement
Day 40	Contributor 2	Propose candidate intervention mechanisms.	Method candidates
Day 41	Mihika	Select proposed-method direction jointly; define hypothesis.	Method hypothesis
Day 41	Contributor 2	Draft implementation design and objective.	Method design
Day 42	Mihika	Implement evaluation hooks for proposed method.	Proposed evaluator
Day 42	Contributor 2	Implement proposed unlearning method.	Proposed method v1
Day 43	Mihika	Run proposed-method smoke test.	Method smoke result

Day 43	Contributor 2	Debug training and checkpoint behavior.	Method debugged
Day 44	Mihika	Run proposed method on 1% forget set.	Proposed 1%
Day 44	Contributor 2	Monitor and log training behavior.	Training log
Day 45	Mihika	Run proposed method on 5%.	Proposed 5%
Day 45	Contributor 2	Evaluate retention and robustness.	Proposed eval
Day 46	Mihika	Run proposed method on 10%.	Proposed 10%
Day 46	Contributor 2	Compare compute/stability with baselines.	Efficiency comparison
Day 47	Mihika	Run direct/paraphrase/indirect evaluation.	Robustness results
Day 47	Contributor 2	Run relearning evaluation.	Relearning results
Day 48	Mihika	Run blast-radius evaluation.	Blast-radius result
Day 48	Contributor 2	Cross-check proposed-method metrics.	Method audit
Day 49	Mihika	Design ablation matrix.	Ablation plan
Day 49	Contributor 2	Implement component-wise ablations.	Ablation code
Day 50	Mihika	Run ablation 1.	Ablation 1
Day 50	Contributor 2	Run ablation 2.	Ablation 2
Day 51	Mihika	Run hyperparameter sensitivity.	Sensitivity table
Day 51	Contributor 2	Repeat key experiment/seed.	Replication result
Day 52	Mihika	Analyze ablation outcomes.	Ablation analysis
Day 52	Contributor 2	Verify statistical calculations.	Stats audit
Day 53	Mihika	Select final experiments for second model.	Generalization plan
Day 53	Contributor 2	Set up second model and compatibility tests.	Second-model setup
Day 54	Mihika	Run baseline on second model if feasible.	Second-model baseline
Day 54	Contributor 2	Run proposed method smoke test.	Second-model method test
Day 55	Mihika	Run final proposed-method validation.	Generalization results
Day 55	Contributor 2	Evaluate robustness and retention.	Generalization evaluation
Day 56	Mihika	Run final blast-radius comparison.	Generalization blast radius
Day 56	Contributor 2	Audit cross-model differences.	Cross-model audit
Day 57	Mihika	Consolidate raw results and metadata.	Results archive
Day 57	Contributor 2	Freeze final code/config versions.	Code freeze candidate
Day 58	Mihika	Generate publication-quality figures.	Figure set v1
Day 58	Contributor 2	Generate publication-quality tables.	Table set v1
Day 59	Mihika	Perform statistical analysis and confidence intervals where feasible.	Stats package
Day 59	Contributor 2	Re-run any failed/unstable experiments.	Rerun package
Day 60	Mihika	Create final comparison matrix.	Final comparison
Day 60	Contributor 2	Create compute/runtime summary.	Efficiency summary
Day 61	Mihika	Draft Introduction and Motivation.	Paper Intro
Day 61	Contributor 2	Draft Related Work.	Related Work draft
Day 62	Mihika	Draft Methodology and research question.	Methods draft
Day 62	Contributor 2	Draft Experimental Setup.	Experiments draft
Day 63	Mihika	Draft Results section.	Results draft
Day 63	Contributor 2	Draft ablation/generalization	Discussion draft

		discussion.	
Day 64	Mihika	Write blast-radius and robustness analysis.	Analysis draft
Day 64	Contributor 2	Write limitations and threats to validity.	Limitations draft
Day 65	Mihika	Integrate full manuscript.	Full draft v1
Day 65	Contributor 2	Technical consistency review.	Technical review
Day 66	Mihika	Revise novelty claims to match evidence.	Novelty revision
Day 66	Contributor 2	Verify every experimental claim against logs/tables.	Claim audit
Day 67	Mihika	Improve figures, captions, and tables.	Figures v2
Day 67	Contributor 2	Improve reproducibility documentation.	Reproducibility package
Day 68	Mihika	Run final code from clean environment where feasible.	Reproducibility test
Day 68	Contributor 2	Verify repository structure and README.	Repository QA
Day 69	Mihika	Write abstract and conclusion.	Abstract/conclusion
Day 69	Contributor 2	Review abstract/conclusion for unsupported claims.	Final claim review
Day 70	Mihika	Full manuscript revision pass.	Paper draft v2
Day 70	Contributor 2	Full technical revision pass.	Technical draft v2
Day 71	Mihika	Prepare supplementary material.	Supplement v1
Day 71	Contributor 2	Prepare experiment/config appendix.	Appendix v1
Day 72	Mihika	Finalize main figures and tables.	Final figures/tables
Day 72	Contributor 2	Final code and artifact cleanup.	Release candidate
Day 73	Mihika	Internal mock peer review.	Review comments
Day 73	Contributor 2	Internal mock peer review from implementation perspective.	Review comments
Day 74	Mihika	Address review comments.	Revision v3
Day 74	Contributor 2	Address technical issues.	Revision v3
Day 75	Mihika	Check formatting, references, equations, captions.	Formatting pass
Day 75	Contributor 2	Check references and reproducibility details.	Reference audit
Day 76	Mihika	Final statistical/experimental consistency check.	Consistency report
Day 76	Contributor 2	Final code-to-paper consistency check.	Code-paper audit
Day 77	Mihika	Prepare submission package.	Submission package
Day 77	Contributor 2	Prepare repository release/tag candidate.	Release candidate
Day 78	Mihika	Final read-through and proofread.	Final manuscript
Day 78	Contributor 2	Final technical proofread.	Final technical manuscript
Day 79	Mihika	Create final PDF and supplementary files.	Submission-ready files
Day 79	Contributor 2	Verify all artifacts open/run and are documented.	Artifact checklist
Day 80	Mihika	Final supervisor/mentor feedback incorporation.	Final paper
Day 80	Contributor 2	Final supervisor/mentor feedback incorporation.	Final code
Day 81	Mihika	Prepare presentation/poster if required.	Presentation draft
Day 81	Contributor 2	Prepare technical demo/results walkthrough.	Demo draft
Day 82	Mihika	Final repository organization and documentation.	Public repo candidate

Day 82	Contributor 2	Tag release candidate and document experiment commands.	Reproducibility release
Day 83	Mihika	Final submission checklist.	Submission checklist
Day 83	Contributor 2	Final independent audit.	Independent audit
Day 84	Mihika	Submit paper / finalize target venue package.	Submission complete
Day 84	Contributor 2	Archive code, configs, results, and notes.	Research archive

12. Git and Collaboration Workflow

The repository should use main as the stable branch. Mihika works on branch 'mihika'; the second contributor should use a separate contributor branch. Features should be committed in small logical units and merged through pull requests after review.

- git checkout -b <feature-name>
- git add .
- git commit -m "descriptive message"
- git push -u origin <feature-name>

Large generated artifacts such as .venv, model checkpoints, processed datasets, and raw experiment outputs should remain excluded by .gitignore. Store reproducibility metadata, scripts, configs, and small result summaries in Git.

13. Final Deliverables

- Reproducible TOFU preparation pipeline.
- Baseline model and pre-unlearning evaluation pipeline.
- 3–4 baseline unlearning implementations or carefully adapted implementations.
- Robustness evaluation suite: direct, paraphrase, indirect, relearning.
- Semantic collateral-damage / blast-radius evaluation.
- Proposed unlearning method based on diagnosed failure mode.
- Ablation and sensitivity experiments.
- Cross-model validation if compute permits.
- Publication-quality tables, figures, and statistical summaries.
- Clean GitHub repository with reproducibility instructions.
- Conference-style research paper and supplementary material.

14. Risks and Mitigation

Risk	Mitigation
Mac cannot efficiently train larger models	Use TinyLlama for development and Colab/cloud GPU for final experiments.
Baseline code is incompatible with current Transformers	Pin compatible versions or adapt minimally while documenting changes.
Direct forgetting improves but robustness does not	Treat this as a useful finding; use it to motivate the proposed robustness component.
Proposed method has high collateral damage	Tune retention constraints and analyze blast radius rather than hiding the result.

Novelty overlaps with recent literature	Perform a final literature/patent check before freezing the novelty claim.
Experiments are unstable	Use fixed seeds, checkpointing, repeated runs, and report variance.
Too many experiments consume time	Prioritize diagnostic experiments; do not expand baselines without a research reason.

15. Definition of Success

FORGE is successful if it produces a defensible empirical answer to the research question, not merely if it achieves the highest forgetting score. The ideal result is a method that demonstrates stronger target forgetting and robustness while preserving more unrelated knowledge and producing a smaller semantic collateral footprint than relevant baselines.

Even if the proposed method does not outperform every baseline, a carefully controlled finding that reveals when and why unlearning fails—supported by reproducible experiments and clear evaluation—can still form a valuable research contribution.

16. Current Project Status

Component	Status
FORGE project name and research direction	Defined
Git repository and Mihika branch	Set up
Python virtual environment	Set up
TOFU download/load test	Completed
TOFU dataset	Verified: 4,000 examples
1% / 5% / 10% splits	Created with seed 42
TinyLlama loading/generation	Verified: 1,100,048,384 parameters
Pre-unlearning baseline evaluator	Next major implementation
Baseline unlearning methods	Pending
Blast-radius evaluation	Pending
Proposed method	Intentionally not locked yet
Final paper	Planned

17. Immediate Next Steps

1. Commit/push the verified setup to the Mihika branch.
2. Build the baseline evaluator using the actual TOFU forget/retain data.
3. Establish pre-unlearning scores and qualitative outputs.
4. Finalize baseline methods based on compatibility and publication relevance.
5. Begin baseline reproduction before designing the proposed method.